

ARM PrimeCell

RTC (PL031)

Integration Manual

ARM PrimeCell Real Time Clock (PL031) Integration Manual

© Copyright ARM Limited 2001. All rights reserved

Proprietary notice

ARM, the Arm powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product Status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The document number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestions for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Real Time Clock

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	6
1.1	Change history	6
1.2	References	6
1.3	Terms and abbreviations	6
2	SCOPE	7
3	INTRODUCTION	7
4	INTEGRATION	8
4.1	Design Flow Supported	8
4.2	Signal Connectivity	9
4.3	Bus Interconnection	10
4.4	Endianness	10
4.5	Customisation	10
4.6	Synthesis	10
4.6.1	Setup Scripts	10
4.6.2	Command Script	11
4.6.3	Constraint Scripts	11
4.7	Integration with AMBA Micropack components	12
4.8	Interrupt Connection	12
4.9	Clocking	12
4.10	Clock Tree Synthesis	13
4.11	Resets	13
4.12	Reset Buffering	14
4.13	Synchronisers	14
4.14	Integration Testing	15
4.15	Test Methodologies	15
4.16	Functional and Timing Verification	16

5	APPENDIX A	17
5.1	PrimeCell Environment Variables	17

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
1.0,2.0	August 6, 2001	Initial versions, with modifications to include integration testing
3.0	August 23, 2001	Final release

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI0224A-04	ARM PrimeCell RTC (PL031) Technical Reference Manual
3	SC058 DESC 00006	ARM PrimeCell RTC (PL031) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05
7	SCB00 QPRO 0008	PrimeCell Integration Vector Strategy document

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTC	Real Time Clock
RTL	Register Transfer Level
SoC	System On-Chip

2 SCOPE

This document describes how the ARM PrimeCell RTC may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow..

3 INTRODUCTION

The ARM PrimeCell RTC is a reusable soft-IP block that has been developed with the primary aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, please refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell RTC block, refer to the ARM PrimeCell RTC (PL031) Technical Reference Manual [Ref.2].

The ARM PrimeCell RTC (PL031) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell RTC block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example microcontroller based on an ARM processor and low-power, generic design methodology of AMBA. Further information can also be found on the use of BusTalk and TicTalk test vectors for functional verification and integration testing.

The AMBA Interconnection Schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tristate within an AMBA system [Ref.6].

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell Real Time Clock (RTC) has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell RTC block.

An example ASIC design flow is described below.

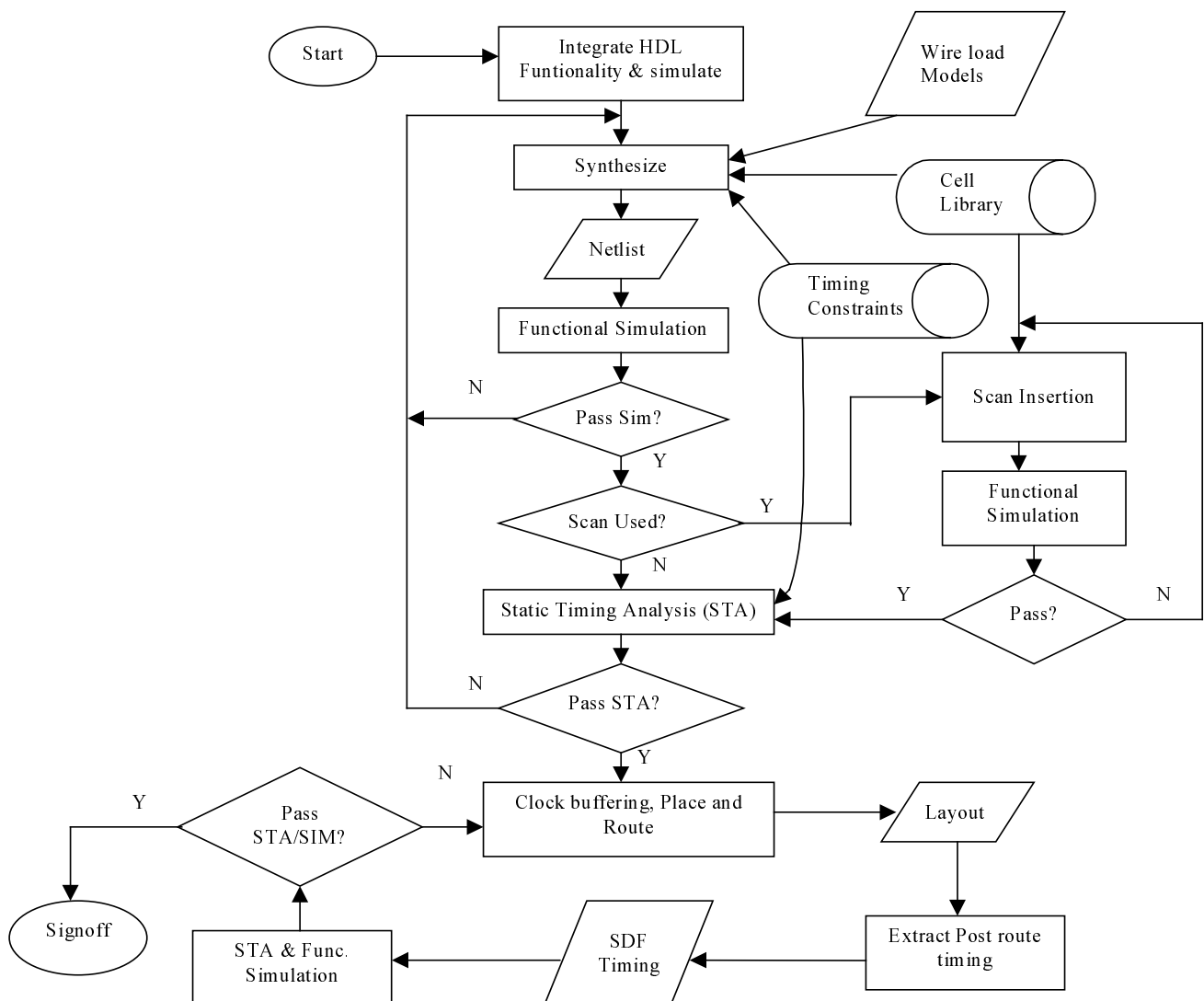


Figure 4-1 Example ASIC Design Flow supported

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell RTC block. For further information refer to the ARM PrimeCell RTC (PL031) Technical Reference Manual [Ref.2].

Table 4-1 APB Signal descriptions

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB Bus Clock.
PRESETn	IN	Reset Controller	Bus Reset (active low).
PENABLE	IN	APB Bridge	This signal is used to time all accesses on the peripheral bus
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the peripheral has been selected by the APB Bridge.
PWRITE	IN	APB Bridge	When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral.
PADDR[11:2]	IN	APB Bridge	This is part of the peripheral address bus, which is used for decoding the internal address space.
PWDATA[31:0]	IN	APB Bridge	Data is written into the peripheral on this bus.
PRDATA[31:0]	OUT	APB Bridge	Data is read out of the peripheral via this bus.

Table 4-2 On-Chip signals

Signal Name	Type	Source / Destination	Description
CLK1HZ	IN	Clock Generator	1Hz clock input. This is the signal that clocks the counter during normal operation.
nPOR	IN	Reset controller	PrimeCell RTC power-on reset signal to Match and Offset registers. They need to retain their values over a bus reset.
nRTCRST	IN	Reset controller	PrimeCell RTC reset signal to CLK1HZ domain, active LOW. The reset controller must use PRESETn to assert nRTCRST asynchronously but negate it synchronously to CLK1HZ.
RTCINTR	OUT	Interrupt Controller	Interrupt signal to the Interrupt module. When HIGH, this signal indicates that a match has occurred between the RTC value and the match register.
SCANENABLE	IN	Test Controller	RTC scan enable signal for both clock domains.
SCANINPCLK	IN	Test Controller	RTC input scan signal for the PCLK domain.
SCANINCLK1HZ	IN	Test Controller	RTC input scan signal for the CLK1HZ domain.
SCANOUTPCLK	OUT	Test Controller	RTC output scan signal for the PCLK domain.
SCANOUTCLK1HZ	OUT	Test Controller	RTC output scan signal for the CLK1HZ domain.

4.3 Bus Interconnection

The RTC block interfaces with AMBA APB interface using separate unidirectional read (output) and write (input) data buses PRDATA and PWDATA. For further information refer to the AMBA Specification [Ref.1].

The data bus is interfaced at the APB Bridge using multiplexers. In common with other APB peripheral blocks, the RTC block forces all read data bus lines low when there are no APB read transfers in progress.

However, it is also possible to interface the RTC block with a 3-state bi-directional data bus. A wrapper needs to be added around the RTC block connection the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the APB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Endianness

The RTC is designed for systems which use either little-endian or big-endian memory organisation.

The 32-bit wide APB data bus is connected to bits 31:0 of the data bus at the APB Bridge. All bus transfers to the RTC block should be done using 32-bit word accesses only.

4.5 Customisation

The ARM PrimeCell RTC can be modified to suit specific user configurations, but the full benefit of a fast integration will only be obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the ARM (PL031) Design Manual [Ref.3].

4.6 Synthesis

The RTC block has been synthesised using the Synopsys Design Compiler synthesis tool with scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script – Rtc.cmd – synthesises the RTC to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Rtc_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and the synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

4.6.1 Setup Scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. [Please refer to Appendix A for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defined the following:

- Synopsys variable settings

- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- target area goals
- maximum transition costs

The library_avanti_cb25_v2.1.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb25_v2.1.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The \$LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

4.6.2 Command Script

The master compile script Rtc.cmd in the synopsys/scripts directory analyses the HDL source before mapping optimising to a specific target technology cell library. The Rtc.cmd script includes set-up and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDT files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script Rtc_scan.cmd in the synopsys/scripts directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables, which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- \$PERIPH is assigned to 'topname' to define the name of the top level module in the design.
- \$HDL_SOURCE is assigned to 'hdl' to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- \$HDL_NETL is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.6.3 Constraint Scripts

- Operating conditions and signal load/drive values should be set for the chosen technology library in the library_avanti_cb25_v2.1.scr file.
- Timing constraints for APB bus signals are set via the apb_params.scr and the apb_slave.scr file. As delivered, the timings are set for a 100 MHz clock. These timings should be adjusted according to the target frequency required. The apb_params.scr file has been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 5% of the clock period.
- the plus uncertainty value is set at 40% of the minus uncertainty value.

The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume a bus-timing budget of 20% for output delays and 40% for input setup. This is a reasonable estimate assuming that the process technology is such that APB Bridge can meet this constraint with signals being available after 40% of the clock period.

If a very high frequency bus clock is being used, it is important to ensure that the read data output timing is adequate to meet the requirements of the APB Bridge. In some designs the read data may not be sampled by the APB Bridge, but instead passed directly to the system bus. In this case, it is necessary to ensure that the timing requirements are met for the block that is sampling the data.

The value of 40% of clock period allowed for input setup of APB inputs applies for a style of APB Bridge design with registered inputs that implement logic after the registers. If the APB Bridge has registered outputs then the input setup can be increased, which should result in a reduction of area required.

- c) Timing constraints for non-AMBA RTC-specific signals are set via the Rtc.scr file. The CLK1HZ frequency is set to be same as that of PCLK for synthesis. Sample input and output delay constraints are also specified for the other signals. It should be noted that for the asynchronous signals, the timing budgets are arbitrary since they can never be accurate. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

4.7 Integration with AMBA Micropack components

The RTC block is designed to be integrated into an AMBA system via the APB (Advanced Peripheral Bus). It is designed to work in systems where an AMBA bus master can read from or write to the RTC through the APB bridge (from AHB) in response to interrupt signals. In the majority of systems, the DMA capability will not be required for a low bandwidth peripheral such as the RTC. It can be efficiently controlled, directly by the CPU.

Typical operation would involve programming a value into the RTC Match Register and initialising the counter with a known value. The RTC would interrupt the CPU via the interrupt controller on a match between the current counter contents and the Match Register. The interrupts should then be cleared by the CPU via an explicit write to the RTC Interrupt Clear Register.

An example APB Bridge is available within the ARM Micropack product. The decode logic within the APB Bridge will need to be altered to take into account the address location of the RTC and any other APB peripheral blocks in the memory map. The base address of the RTC needs to be defined on a system specific basis and the bridge decode logic altered accordingly. The APB Bridge appears as slave block on the AHB bus with a decode signal, which must be altered in the HDL code of the AMBA decoder block, to select the correct range for the APB blocks (including the RTC). If AMBA TIC methodology is to be used for integration testing then any change of address location for the RTC block must also be reflected in the AMBA TIC integration test vectors, by amending the base address declaration for the RTC integration test program.

4.8 Interrupt Connection

The RTC block provides a single interrupt line RTCINTR. This interrupt source can be masked within the RTC itself, thus complementing the mask bit provided in the interrupt controller. The ARM core has two interrupts – nIRQ and nFIQ. Normally a low priority peripheral block such as the RTC would route the interrupt source to the nIRQ input of the ARM core.

4.9 Clocking

The RTC HDL contains two clock sources at the top level – PCLK and CLK1HZ. While it would be beneficial to clock all internal logic off a single clock, this is not possible due to the internal implementation considerations. The APB clock, PCLK, needs to be at least 3 times faster than CLK1HZ.

Static timing analysis will need to be performed on the separate clock domains and two scan chains will be needed during scan insertion.

PCLK can be clocked by the AMBA system clock, BCLK/HCLK, or the APB Bridge could be designed so that PCLK is an integer sub-multiple of HCLK if a lower frequency bus clock is required in order to reduce power consumption.

It should be noted that the RTC design assumes that the CLK1HZ is free running during normal device operation. On the other hand, shutting off PCLK does not affect the generation of the RTC interrupt as this occurs in the CLK1HZ domain. In addition the match and offset values required for generating the RTC interrupt are retained over a PCLK domain shut-down to allow the interrupt trigger the device back into operation during such a shut down if asserted. It should also be noted that a CLK1HZ system reset would require a reprogramming of the RTC via the appropriate registers.

These design measures should cater for the possible need to gate PCLK in very power critical applications. If gating is required, it should be implemented in a separate block to ensure ease of integration of the RTC block in a typical design flow.

4.10 Clock Tree Synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default RTC synthesis scripts cause the Synopsys tool to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place and route tools that understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during place and route stage and not during synthesis.

4.11 Resets

The RTC block has one reset pin for each of its internal clock domains. In addition it has a power-on reset signal, nPOR. The system bus reset signal PRESETn, resets the PCLK domain registers. The nPOR power-on reset signal resets the match and offset registers as they need to retain their values over a PCLK system reset since both values are required for generating the RTC interrupt. In addition, the nPOR reset signal has to be synchronously de-asserted to the system clock, PCLK, before use. This should be taken care of by the system designer.

A separate reset pin nRTCRST is used to reset the CLK1HZ domain registers. This signal needs to be provided by a separate reset controller, and it must be de-asserted synchronously to CLK1HZ.

The nRTCRST signal should only be used during a hard reset, while the PRESETn signal can be used during a hard and soft reset.

Figure 4-2 A method of generating nRTCRST illustrates a method of generating nRTCRST from PRESETn, such that it will be asserted asynchronously and de-asserted synchronously to CLK1HZ. The reset signal nRTCRST is de-asserted two CLK1HZ cycles after PRESETn is de-asserted to prevent metastability problems.

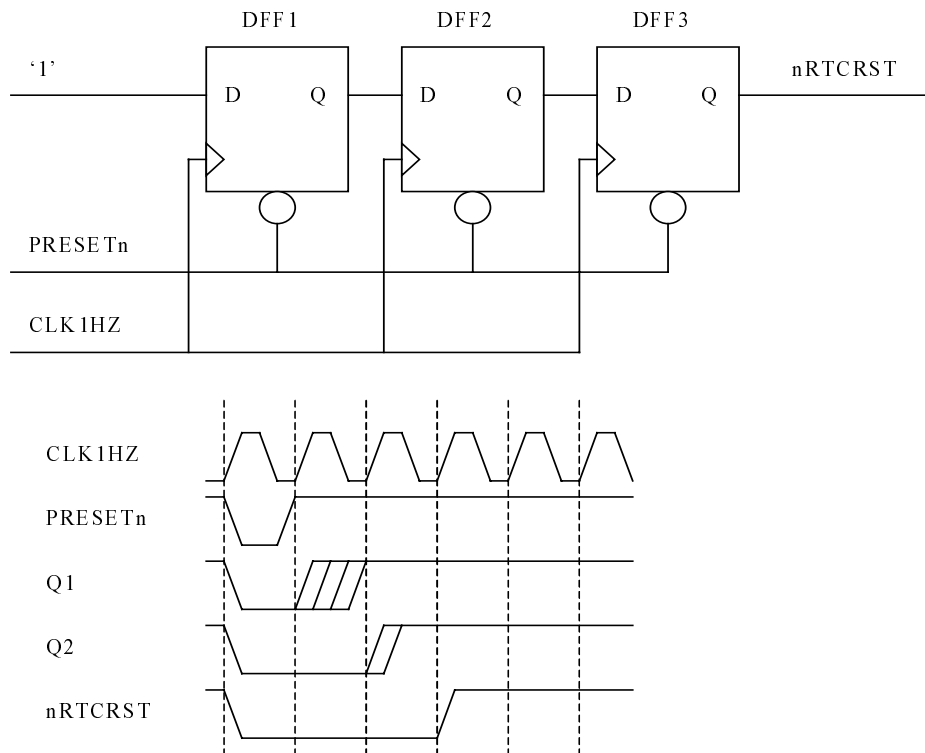


Figure 4-2 A method of generating nRTCRST

This method of generating nRTCRST could experience simulation problems depending on the cell library modelling. The Q output of DFF3 should not be metastable when setup/hold violations occurs due to de-assertion of PRESETn, since both D and Q are both '0', however the DFF model may assign 'X'.

4.12 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset net. The reset signal could be assigned infinite drive strength allowing a chip-level integration process to ensure a balanced reset tree is used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

4.13 Synchronisers

Single bit signals between the two clock domains are synchronised by two series flip-flops inferred during synthesis from standard HDL templates. Multiple bit vector signals use an update mechanism with synchronised single bit control signals to avoid metastability problems.

Some technology libraries offer metastability hardened D flip-flops specifically for implementing synchronisers. All the synchronisers in the RTC block have been isolated to the sub-block RtcSynctoPCLK. This allows the synthesis scripts to be modified so that these blocks could be synthesised separately to use the synchroniser flip-flop cells using the Synopsys Design Compiler set_prefer command. Further synthesis of these sub-block can then be prevented using the set_don't_touch command. The rest of the RTC design is then synthesised after placing a set_don't_use command on the synchroniser cells. This method also offers the opportunity to simulate the gate-level netlist with asynchronous clocks PCLK and CLK1HZ and eliminate the expected setup and hold

violations on the synchronisers from the simulation transcript, since the synchroniser cells can be modified to remove 'X' propagation.

4.14 Integration Testing

The RTC integration test vector suite allows the integrator to verify that the RTC has been connected into the target system correctly. These vectors test 3 classes of signals.

- a) AMBA signals (RTC signals connected directly to the APB)
- b) Intra-Chip signals (RTC signals connected to the Interrupt Controller)
- c) Primary I/O signals (RTC signals connected to the chip pads – none for the RTC)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref.7].

The integration vectors are contained in the `Rtc_Integ.c` file in the integration/bustest directory. As delivered, these vectors are intended to run correctly with the RTC in stand-alone mode i.e. even before the RTC has been integrated into the system. After integration, the following points need to be noted:

- a) The integration vectors targeting the RTC to APB (AMBA signals) connectivity do **NOT** have to be modified by the integrator.
- b) The integration vectors targeting the RTC to Interrupt Controller (Intra-Chip signals) will NEED modification. The `Rtc_Integ.c` file contains notes listing the changes that the integrator would have to make in the test code.

4.15 Test Methodologies

The RTC has been designed such that it can be tested using scan-insertion and ATPG.

It is assumed the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The RTC design fully supports the scan test and ATPG methodology. Based on the clocking arrangement used (see section 4.9), it will be necessary to use two scan chains. Only the rising edges of clocks are used in this design. As delivered, the RTC is synthesised with two scan chains, one each for the PCLK and CLK1HZ domains. By having dedicated scan chains for each clock domain, complete freedom is given to the integrator to decide upon chip level scan-chain ordering.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through the use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with scan test D flip-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan insertion and ATPG approach is in cases where significant modifications have been made to the RTC block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that very high fault coverage can be achieved (99%).

The system designer performs extraction of test vectors once the whole chip design has been integrated. The quoted fault coverage (except variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running either the TIC source or the automatically generated vectors

(ATPG). In the case of TIC, only the 36 TIC port signals need to be controlled/monitored by the tester to achieve the required fault coverage.

4.16 Functional and Timing Verification

It is essential to verify timing performance of the netlist against the RTL at key stage in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC.

Gate-level simulation can be used for limiting timing verification but it is slow and cannot exercise all timing paths in design. Results tend to be optimistic since worst case timing path may not be exercised.

Note: There are known Verilog-XL SDF annotation errors associated with LDV simulations for RTL and gate-level netlists. They do not affect the simulation results.

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis. Use of gate-level simulation as well as Static Timing Analysis is a useful check for human error in the manual definition of false paths.

The netlist should also be checked for functional equivalence to the RTL at the same key stages of the design flow.

This can be performed by dynamic simulation using test vectors as described above, or by use of a logic equivalence-checking tool (For example, Synopsys Formality or Chrysalis).

5 APPENDIX A

5.1 PrimeCell Environment Variables

Simulation and Synthesis in the Rtc world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the '**GLOBAL**' variable.

E.g. : **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the '**PERIPH**' variable, which should be created as an empty variable.

E.g. : **setenv PERIPH <enter>**

the variable is then set to '**Rtc**' in the RTC makefile, this enables the make utility to change the variable value for each peripheral in the design.

SIMULATOR

The simulator type is defined by the '**SIMULATOR**' variable.

It can take the following two values

- Modelsim - for ModelSim simulations
- LDV - for LDV simulations

e.g. : **setenv SIMULATOR modelsim**

setenv SIMULATOR LDV

TEST_METH

The scan methodology adopted during synthesis is defined by the '**TEST_METH**' variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scanready - creates a netlist with scan F/Fs but without scan routing;
- scaninsert - creates a fully routed scan netlist.

e.g. : **setenv TEST_METH noscan**
 setenv TEST_METH scanready
 setenv TEST_METH scaninsert

HDL_SOURCE

The source code type is defined by the '**HDL_SOURCE**' variable.

It can take the following values

- vhdl [VHDL RTL]
- verilog[Verilog RTL]

e.g. : **setenv HDL_SOURCE vhdl**
 setenv HDL_SOURCE verilog

HDL_NETL

The netlist type is defined by the '**HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog[Verilog netlist]

e.g. : **setenv HDL_NETL vhdl**
 setenv HDL_NETL verilog

TEST_ENV

The test environment is defined by the '**TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

e.g. : **setenv TEST_ENV BUSTALK**
 setenv TEST_ENV TICTALK

LIB_VHDL

This location of the vhdl cell library views is defined by the '**LIB_VHDL**' variable.

e.g. : **setenv LIB_VHDL /AVANT/1999.1/cb25/v2.1/vital/cb25os163/zero**

DIRS_VERILOG

This location of the verilog cell library views is defined by the '**DIRS_VERILOG**' variable.

e.g. : **setenv DIRS_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero**

FILES_VERILOG

This location of the verilog UDP file is defined by the '**FILES_VERILOG**' variable.

e.g : **setenv FILES_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v**

LIB_SYNOP

This location of the synopsys target library files is defined by the '**LIB_SYNOP**' variable.

e.g : **setenv LIB_SYNOP /AVANT/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models**